

Ex. No: 12 Page Replacement Algorithms

PROGRAM A: FIFO PAGE REPLACEMENT

C PROGRAM

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];
    int n, f, i, j, k = 0;
    int fault = 0, found;

    printf("Enter Number of Pages: ");
    scanf("%d", &n);

    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter Number of Frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] =
        -1;

    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
    }
```

```

if(found == 0)
{
frames[k] = pages[i];
k = (k + 1) % f;
fault++;
}
}

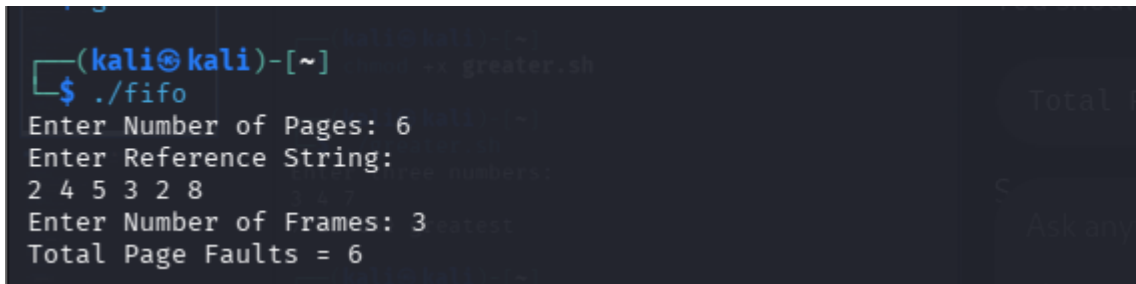
printf("Total Page Faults = %d\n", fault);

return 0;

}

```

Output:



```

(kali㉿kali)-[~]
$ ./fifo
Enter Number of Pages: 6
Enter Reference String:
2 4 5 3 2 8
Enter Number of Frames: 3
Total Page Faults = 6

```

SHELL SCRIPT

```

#!/bin/bash

echo "FIFO Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

frames=3

echo "Reference String: ${pages[@]}"

echo "Frames: $frames"

echo "FIFO Algorithm Executed"

```

Output:

```

(kali㉿kali)-[~]
└─$ chmod +x fifo.sh
(kali㉿kali)-[~]
└─$ ./fifo.sh
FIFO Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames: 3
FIFO Algorithm Executed

```

PROGRAM B: LRU PAGE REPLACEMENT

C PROGRAM

```

#include <stdio.h>

int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j;
    int fault = 0, count = 0;
    int found, pos, min;
    printf("Enter Number of Pages: ");
    scanf("%d", &n);
    printf("Enter Reference String:
\n");

    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++)
        frames[i] =
        -1;
    for(i = 0; i < n; i++)
    {
        found = 0;

```

```
for(j = 0; j < f; j++)
{
    if(frames[j] == pages[i])
    {
        count++;
        time[j] = count;
        found = 1;
        break;
    }
}
if(found == 0)
{
    min = time[0];
    pos = 0;
    for(j = 0; j < f; j++)
    {if(frames[j] == -1)
    {
        pos = j;
        break;
    }
    if(time[j] < min)
    {
        min = time[j];
        pos = j;
    }
}
frames[pos] = pages[i];
count++;
time[pos] = count;
fault++;
}
```

```

}

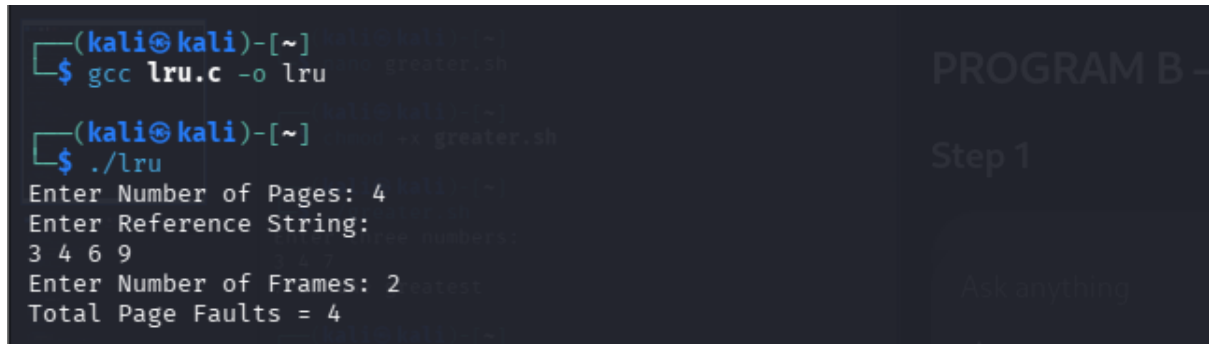
printf("Total Page Faults = %d\n", fault);

return 0;

}

```

Output:



```

(kali㉿kali)-[~]
└─$ gcc lru.c -o lru
(kali㉿kali)-[~]
└─$ ./lru
Enter Number of Pages: 4
Enter Reference String:
3 4 6 9
Enter Number of Frames: 2
Total Page Faults = 4

```

SHELL SCRIPT

```

#!/bin/bash

echo "LRU Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

echo "Reference String: ${pages[@]}"

echo "LRU Algorithm Executed"

```

Output:



```

(kali㉿kali)-[~]
└─$ chmod +x lru.sh
(kali㉿kali)-[~]
└─$ ./lru.sh
LRU Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
LRU Algorithm Executed

```

PROGRAM C: OPTIMAL PAGE REPLACEMENT

C PROGRAM

```

#include <stdio.h>

int main()

```

```

{
int pages[50], frames[10];

int n, f;

int i, j, k, pos;

int fault = 0;

int found;

printf("Enter Number of Pages: ");

scanf("%d", &n);

printf("Enter Reference String:
\n");


for(i = 0; i < n; i++)
scanf("%d", &pages[i]);

printf("Enter Number of Frames: ");

scanf("%d", &f);

for(i = 0; i < f; i++)

frames[i] =

-1;

for(i = 0; i < n; i++)

{

found = 0;

for(j = 0; j < f; j++)

{

if(frames[j] == pages[i])

{

found = 1;

break;

}

}

if(found == 0)

{

```

```

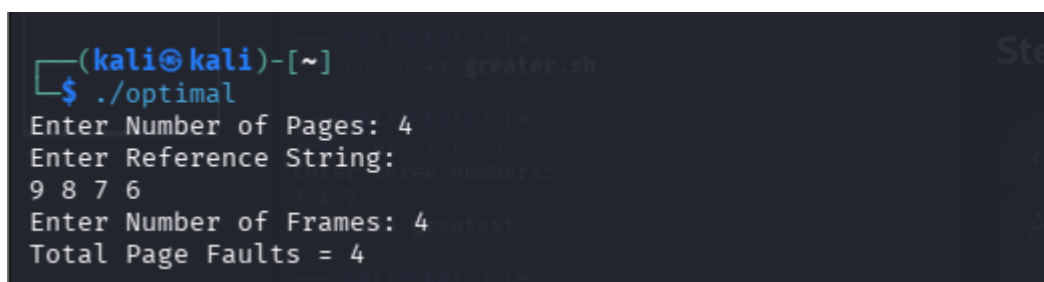
for(j = 0; j < f; j++)
{
    int future = 999;
    for(k = i + 1; k < n; k++)
    {
        if(frames[j] == pages[k])

        {

            future = k;
            break;
        }
    }
    if(j == 0 || future > pos)
    {
        pos = future;
    }
}
frames[0] = pages[i];
fault++;
}
}
printf("Total Page Faults = %d\n", fault);
return 0;
}

```

Output:



```

kali@kali:~$ ./optimal
Enter Number of Pages: 4
Enter Reference String: 9 8 7 6
Enter Number of Frames: 4
Total Page Faults = 4

```

SHELL SCRIPT

```
#!/bin/bash
```

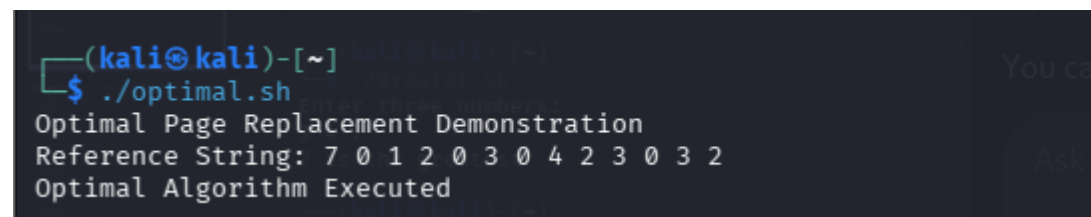
```
echo "Optimal Page Replacement Demonstration"
```

```
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
```

```
echo "Reference String: ${pages[@]}"
```

```
echo "Optimal Algorithm Executed"
```

Output:

A terminal window screenshot with a dark background. The prompt is (kali㉿kali)-[~]. The user enters ./optimal.sh. The script outputs three lines: "Optimal Page Replacement Demonstration", "Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2", and "Optimal Algorithm Executed".

```
(kali㉿kali)-[~]  
$ ./optimal.sh  
Optimal Page Replacement Demonstration  
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2  
Optimal Algorithm Executed
```